EXP NO:1	8051 Assembly Language program using Keil simulator
DATE	

AIM:

To write 8051 Assembly Language Program for an 8-bit addition using Keil simulator and execute it.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keil µvision5 IDE	1

INTRODUCTION TO 8051 SIMULATORS:

A simulator is software that will execute the program and show the results exactly to the program running on the hardware, if the programmer finds any errors in the program while simulating the program in the simulator, he can change the program and re-simulate the code and get the expected result, before going to the hardware testing. The programmer can confidently dump the program in the hardware when he simulates his program in the simulator and gets the expected results.

8051 controller is a most popular 8-bit controller which is used in a large number of embedded applications and many programmers write programs according to their application. So testing their programs in the software simulators is a way. Simulators will help the programmer to understand the errors easily and the time taken for the testing is also decreased.

These simulators are very useful for students because they do need not to build the complete hardware for testing their program and validate their program very easily in an interactive way.

List of 8051 Simulators:

The list of simulators is given below with their features:

1. MCU 8051: MCU 8051 is an 8051 simulator that is very simple to use and has an interactive IDE (Integrated Development Environment). It is developed by Martin Osmera and most important of all is that it is completely free. There are many features for this IDE they are

- ✓ It supports both C and assembly language for compilation and simulation

- ✓ It has an in-built source code editor, graphical notepad, ASCII charts, Assembly symbol viewer, etc. It also supports several 8051 ICs like at89c51, A89S52, 8051, 8052, etc.
- ✓ It will support certain electronic simulations like LED, 7segment display, LCD etc. which will help in giving the output when you interface these things to the hardware directly.
- ✓ It has tools like hex decimal editors, base converters, special calculator, file converters, inbuilt hardware programmers, etc.
- ✓ It has syntax validation, pop base auto-completion etc.

You can download this tool from https://sourceforge.net/projects/mcu8051ide/files/.

2. EDSIM 51: This is a virtual 8051 interfaced with virtual peripherals like 7 segment display, motor, keypad, UART etc. This simulator is exclusively for students developed by James Rogers,. The features of this simulator are

- ✓ Have virtual peripherals like ADC, DAC with scope to display, comparator etc.
- ✓ Supports only assembly language
- ✓ IDE is completely written in JAVA and supports all the OS.
- ✓ Completely free and with user guide, examples, etc.

You can download this simulator from the https://www.edsim51.com/index.html.

3. 8051 IDE: This simulation software is exclusively for the Windows operating system (98/xp). The features of this simulator are

- ✓ Text editor, assembler, and software simulate in one single program.
- ✓ Has facilities like Breakpoint setter, execute to break point, predefined simulator watch window, etc.
- ✓ It is available in both free version and paid version.

You can download this tool from https://www.acebus.com/win8051.htm

4. KEIL µVision: KEIL is the most popular software simulator. It has many features like interactive IDE and supports both C and assembly languages for compilation and simulation.

You can download and get more information from https://www.keil.com/c51/.

INSTALLATION OF KEIL SOFTWARE

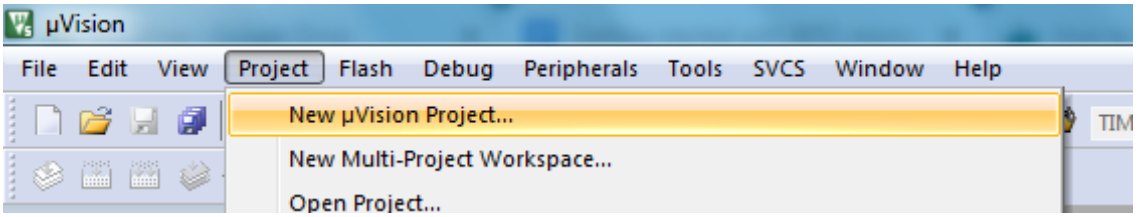
Set up Keil IDE for Programming

Keil µVision IDE is a popular way to program MCUs containing the 8051 architectures. It supports over 50 microcontrollers and has good debugging tools including logic analyzers and watch windows. In this article, we will use the AT89C51ED2 microcontroller, which has:

- 64 KB FLASH ROM
- On-chip EEPROM
- 256 Bytes RAM
- In-System programming for uploading the program
- 3 Timer/counters
- SPI, UART, PWM

The Keil µVision icon.

To start writing a new program, you need to create a new project. Navigate to **project —> New µVision project**. Then save the new project in a folder.



After saving the file, a new window will pop up asking you to select your microcontroller.

As discussed, we are using AT89C51/AT89C51ED2/AT89C52, so select this controller under the Microchip section (as Atmel is now a part of Microchip)

Next, we have to add that .c or .asm file to our project workspace. Select **Add Existing** Files and then select the created .c or .asm file to get it added.

The workspace and project file are ready.

PROCEDURE

1. Create a new project, go to “Project” and close the current project “Close Project”.
2. Next Go to the Project New µVision Project and Create New Project Select Device for Target.
3. Select the device AT89C51ED2 or AT89C51 or AT89C52
4. Add Startup file Next go to “File” and click “New”.
5. Write a program on the editor window and save it with .asm extension.
6. Add this source file to Group and click on “Build Target” or F7.
7. Go to debugging mode to see the result of simulation by clicking Run or step run.

PROGRAM:

```
ORG 0000H
CLR C
MOV A, #20H
ADD A, #21H
MOV R0, A
END
```

EXP NO:2	Test data transfer between registers and memory
DATE	

AIM:

To write and execute an Assembly language program to transfer data between registers and memory.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keil µvision5 IDE	1

PROCEDURE

1. Create a new project, go to “Project” and close the current project “Close Project”.
2. Next Go to the Project New µVision Project and Create a New Project Select Device for the Target.
3. Select the device AT89C51ED2 or AT89C51 or AT89C52
4. Add Startup file Next go to “File” and click “New”.
5. Write a program on the editor window and save it with .asm extension.
6. Add this source file to Group and click on “Build Target” or F7.
7. Go to debugging mode to see the result of the simulation by clicking Run or Step run.

PROGRAM:

TYPE-I:

```
ORG 0000H
CLR C
MOV R0, #55H
MOV R1, #6FH
MOV A, R0
MOV 30H, A
MOV A, R1
MOV 31H, A
END
```

PROGRAM:

TYPE-II:

```
        ORG 0000H
        CLR C
        MOV R0, #30H
        MOV R1, #40H
        MOV R7, #06H
BACK: MOV A, @R0
        MOV @R1, A
        INC R0
        INC R1
        DJNZ R7, BACK
        END
```

EXP NO:3	ALU operations
DATE	

AIM:

To write and execute the ALU program using the Keil simulator.

SOFTWARE TOOLS REQUIRED:

S.No	Software Requirements	Quantity
1	Keil µvision5 IDE	1

PROCEDURE

1. Create a new project, go to “Project” and close the current project “Close Project”.
2. Next Go to the Project New µVision Project and Create New Project Select Device for Target.
3. Select the device AT89C51ED2 or AT89C51 or AT89C52
4. Add Startup file Next go to “File” and click “New”.
5. Write a program on the editor window and save it with .asm extension.
6. Add this source file to Group and click on “Build Target” or F7.
7. Go to debugging mode to see the result of simulation by clicking Run or step run.

PROGRAM:

```
ORG 0000H
CLR C
//ADDITION
MOV A, #20H
ADD A, #21H
MOV 41H, A

//SUBTRACTION
MOV A, #20H
SUBB A, #18H
MOV 42H, A

//MULTIPLICATION
MOV A, #03H
MOV B, #04H
MUL AB
MOV 43H, A

//DIVISION
MOV A, #95H
MOV B, #10H
DIV AB
MOV 44H, A
MOV 45H, B

//AND
MOV A, #25H
MOV B, #12H
ANL A, B
MOV 46H, A

//OR
MOV A, #25H
MOV B, #15H
ORL A, B
MOV 47H, A

//XOR
MOV A, #45H
MOV B, #67H
```

```
XRL A, B
MOV 48H, A

//NOT
MOV A, #45H
CPL A
MOV 49H, A
END
```

EXP NO:4(a)	WRITE BASIC PROGRAMS USING EMBEDDED C
DATE	

AIM:

To write a basic embedded C program to control a port 0 pin 0 connected to an 8051 microcontroller using a Keil simulator.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keil µvision5 IDE	1

PROCEDURE

1. Create a new project, go to “Project” and close the current project “Close Project”.
2. Next Go to the Project New µVision Project and Create a New Project Select Device for the Target.
3. Select the device AT89C51ED2 or AT89C51 or AT89C52
4. Add Startup file Next go to “File” and click “New”.
5. Write a program on the editor window and save it with .asm extension.
6. Add this source file to Group and click on “Build Target” or F7.
7. Go to debugging mode to see the result of the simulation by clicking Run or Step run.

PROGRAM:

```
#include<REG51.h>
int i,j;
sbit LED = P2^0;
void main()
{
  while(1)
    {
        LED = 0;
        for(j=0;j<10000;j++);
        LED = 1;
        for(j=0;j<10000;j++);
    }
}
```

EXP NO:4(b)	ARITHMETIC PROGRAMS USING EMBEDDED C
DATE	

AIM:

To write an embedded C program for addition, subtraction, multiplication, and division using the Keil simulator.

SOFTWARE REQUIRED:

S.No	Software Requirements	Quantity
1	Keil µvision5 IDE	1

PROCEDURE

1. Create a new project, go to “Project” and close the current project “Close Project”.
2. Next Go to the Project New µvision Project and Create New Project Select Device for Target.
3. Select the device AT89C51ED2 or AT89C51 or AT89C52
4. Add Startup file Next go to “File” and click “New”.
5. Write a program on the editor window and save it with the .asm extension.
6. Add this source file to Group and click on “Build Target” or F7.
7. Go to debugging mode to see the result of the simulation by clicking Run or Step run.

PROGRAM:

```
#include<REG51.H>
unsigned char a, b;
void main()
{
        a=0x10;
        b=0x04;
        P0=a-b;
        P1=a+b;
        P2=a*b;
        P3=a/b;
while(1);
}
```

OUTPUT:

Port P0= _____

Port P1= _____

Port P2= _____

Port P3= _____

EXP NO:5	INTRODUCTION TO THE ARDUINO PLATFORM
DATE	

AIM:

To study the basics of Arduino Uno board and Arduino IDE 2.0 software.

Hardware & SOFTWARE TOOLS REQUIRED:

S.No.	Hardware & Software Requirements	Quantity
1	Arduino IDE 2.0	1
2	Arduino Uno Board	1

INTRODUCTION TO ARDUINO:

Arduino is a project, open-source hardware, and software platform used to design and build electronic devices. It designs and manufactures microcontroller kits and single-board interfaces for building electronics projects. The Arduino boards were initially created to help students with the non-technical background. The designs of Arduino boards use a variety of controllers and microprocessors. Arduino is an easy-to-use open platform for creating electronic projects. Arduino boards play a vital role in creating different projects. It makes electronics accessible to non-engineers, hobbyists, etc.

The various components present on the Arduino boards are a Microcontroller, Digital Input/output pins, USB Interface and Connector, Analog Pins, reset buttons, Power buttons, LEDs, Crystal oscillators, and Voltage regulators. Some components may differ depending on the type of board. The most standard and popular board used over time is Arduino UNO. The ATmega328 Microcontroller present on the UNO board makes it rather powerful than other boards. There are various types of Arduino boards used for different purposes and projects. The Arduino Boards are organized using the Arduino (IDE), which can run on various platforms. Here, IDE stands for Integrated Development Environment. Let's discuss some common and best Arduino boards.

EXP NO:7	INTRODUCTION TO ARDUINO PROGRAMMING
DATE	

AIM:

To write and execute different Arduino programming for analog, digital signals and serial communication.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Arduino IDE 2.0	1
2	Arduino UNO Development Board	1
3	Jumper Wires	few
4	Arduino USB Cable	1
5	Joystick Module	1

CONNECTION:

Arduino UNO Pin	Arduino Development Board
2	**LED**

PROGRAM:

DIGITAL WRITE:

```
void setup() {
  pinMode(2, OUTPUT);
}
void loop() {
  digitalWrite(2, HIGH);
  delay(1000);
  digitalWrite(2, LOW);
  delay(1000);
}
```

CONNECTION:

Arduino UNO Pin	Arduino Development Board
2	**LED**
5	**S1 (SW 1)**

DIGITAL READ:

```
void setup() {
  pinMode(2, OUTPUT);
  pinMode(5, INPUT_PULLUP);
}
void loop() {
  int sw=digitalRead(5);
  if(sw==1)
  {
    for(int i=0; i<5; i++)
    {
      digitalWrite(2, HIGH);
      delay(1000);
      digitalWrite(2, LOW);
      delay(1000);
    }
  }
  else
  {
    digitalWrite(2, LOW);
  }
}
```

CONNECTION:

Arduino UNO Pin	Arduino Development Board	Joystick Module
2	**LED**	
VCC or 5V		**+5V**
GND		**GND**
A0		**VRx or VRy**

ANALOG READ:

```
void setup() {
  pinMode(2, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int joystick=analogRead(A0);
  Serial.println(joystick);

  if(joystick>800)
  digitalWrite(2, HIGH);
  else
    digitalWrite(2, LOW);

  delay(500);
}
```

CONNECTION:

Arduino UNO Pin	Arduino Development Board
3	**LED**

PWM Pins: 3, 5, 6, 9, 10, 11

ANALOG WRITE:

```
void setup() {
  pinMode(3, OUTPUT);
}
void loop() {
  for(int i=0; i<256;i++) {
    analogWrite(3,i);
    delay(20);
  }
  for(int i=255; i>=0;i--) {
    analogWrite(3,i);
    delay(20);
  }
}
```

CONNECTION:

Arduino UNO Pin	Arduino Development Board
4	**LED**

SERIAL COMMUNICATION:

```
void setup() {
        Serial.begin(9600);
        pinMode(4, OUTPUT);
}

void loop() {
if(Serial.available()>0)
  {
        char data=Serial.read();
        Serial.println(data);
        if(data=='1'){
          digitalWrite(4,HIGH);
        }
         else if(data=='2'){
          digitalWrite(4,LOW);
        }
    }
}
```

EXP NO:6(a)	**Different communication methods with IoT devices (Zigbee, GSM, Bluetooth)**
DATE	

AIM:

To Explore different communication methods with IoT devices (Zigbee, GSM, Bluetooth).

DIFFERENT COMMUNICATION METHODS:

IoT devices require reliable and efficient communication methods to transmit data and interact with other devices or systems. Here are three commonly used communication methods for IoT devices:

Zigbee:

Zigbee is a low-power wireless communication protocol designed for short-range communication between devices. It operates on the 2.4 GHz frequency band and supports mesh networking, allowing devices to communicate with each other through intermediate nodes. Zigbee is commonly used in home automation, industrial control, and smart energy applications.

GSM (Global System for Mobile Communications):

GSM is a widely used cellular network technology that enables IoT devices to connect to the internet using SIM cards. It operates on various frequency bands and provides wide coverage, making it suitable for applications that require long-range communication. GSM is commonly used in applications such as asset tracking, remote monitoring, and smart cities.

Bluetooth:

Bluetooth is a short-range wireless communication technology that operates on the 2.4 GHz frequency band. It is commonly used for connecting IoT devices to smartphones, tablets, and other nearby devices. Bluetooth Low Energy (BLE) is a power-efficient version of Bluetooth that is ideal for battery-powered IoT devices. Bluetooth is widely used in applications such as wearable devices, healthcare monitoring, and home automation.

Each communication method has its advantages and limitations, and the choice depends on the specific requirements of the IoT application. Factors to consider include range, power consumption, data rate, security, and interoperability with other devices or systems

EXP NO:6(b)	**BLUETOOTH COMMUNICATION**
DATE	

AIM:

To write a program to control an LED using a Bluetooth module.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Arduino IDE 2.0	1
2	Arduino UNO Development Board	1
3	Jumper Wires	few
4	Arduino USB Cable	1
5	HC-05 Bluetooth Module	1

CONNECTIONS:

Arduino UNO Pin	Bluetooth Module	Arduino Development Board
VCC	5V	-
GND	GND	-
2	Tx	-
3	Rx	-
4	-	LED

PROGRAM:

```
#include<SoftwareSerial.h>
SoftwareSerial mySerial(2,3); //rx,tx
void setup() {
mySerial.begin(9600);
Serial.begin(9600);
pinMode(4, OUTPUT);
}

void loop() {
if(mySerial.available()>0)
    {
        char data=mySerial.read();
        Serial.println(data);
        if(data=='1'){
        digitalWrite(4,HIGH);
        Serial.println("LED ON");
        }
        else if(data=='2'){
         digitalWrite(4,LOW);
         Serial.println("LED OFF");
        }
    }
```

EXP NO:6(c)	ZIGBEE COMMUNICATION
DATE	

AIM:

To write a program to control an LED using a Zigbee module.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Arduino IDE 2.0	1
2	Arduino UNO Development Board	2
3	Jumper Wires	few
4	Arduino USB Cable	2
5	Zigbee Module	2

CONNECTIONS:

TRANSMITTER:

Arduino UNO Pin	Zigbee Module
VCC	5V
GND	G
2	Tx
3	Rx

PROGRAM:
TRANSMITTER SIDE:

```
#include<SoftwareSerial.h>
SoftwareSerial mySerial(2,3); //rx,tx
void setup() {
	mySerial.begin(9600);
	Serial.begin(9600);
}

void loop() {
	mySerial.write('A');
	Serial.println('A');
	delay(100);
	mySerial.write('B');
	Serial.println('B');
	delay(100);
}
```

CONNECTIONS:

RECEIVER:

Arduino UNO Pin	Zigbee Module	Arduino Development Board
-	5V	5V
-	G	GND
2	Tx	-
3	Rx	-
4	-	LED1

RECEIVER SIDE:

```
#include<SoftwareSerial.h>
SoftwareSerial mySerial(2,3); //rx,tx
void setup() {
        mySerial.begin(9600);
        Serial.begin(9600);
        pinMode(4, OUTPUT);
}

void loop() {
if(mySerial.available()>0)
{
          char data=mySerial.read();
          Serial.println(data);
          if(data=='A')
          digitalWrite(4,HIGH);
          else if(data=='B')
          digitalWrite(4,LOW);
}
}
```

EXP NO:7	INTRODUCTION TO THE RASPBERRY PI PLATFORM
DATE	

Introduction to Raspberry Pi Pico W:

The Raspberry Pi Pico W is a compact and affordable microcontroller board developed by the Raspberry Pi Foundation. Building upon the success of the Raspberry Pi Pico, the Pico W variant brings wireless connectivity to the table, making it an even more versatile platform for embedded projects. In this article, we will provide a comprehensive overview of the Raspberry Pi Pico W, highlighting its key features and capabilities.

Features:

- RP2040 microcontroller with 2MB of flash memory
- On-board single-band 2.4GHz wireless interfaces (802.11n)
- Micro USB B port for power and data (and for reprogramming the flash)
- 40 pins 21mmx51mm 'DIP' style 1mm thick PCB with 0.1″ through-hole pins also with edge castellations
- Exposes 26 multi-function 3.3V general purpose I/O (GPIO)
- 23 GPIO are digital-only, with three also being ADC-capable
- Can be surface mounted as a module
- 3-pin ARM serial wire debug (SWD) port
- Simple yet highly flexible power supply architecture
- Various options for easily powering the unit from micro-USB, external supplies, or batteries
- High quality, low cost, high availability
- Comprehensive SDK, software examples, and documentation
- Dual-core Cortex M0+ at up to 133MHz
- On-chip PLL allows variable core frequency
- 264kByte multi-bank high-performance SRAM

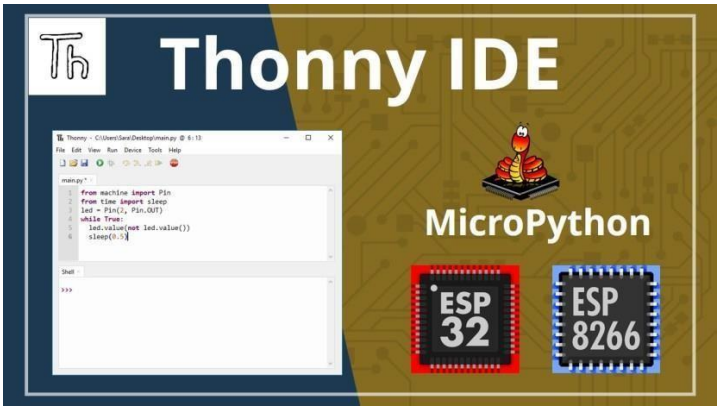
RP2040 Microcontroller:

At the core of the Raspberry Pi Pico W is the RP2040 microcontroller. It features a dual-core ARM Cortex-M0+ processor running at 133MHz, providing ample processing power for a wide range of applications. The microcontroller also includes 264KB of SRAM, which is essential for storing and manipulating data during runtime. Additionally, the RP2040 incorporates 2MB of onboard fla

EXP NO:8	INTRODUCTION TO PYTHON PROGRAMMING
DATE	

Getting Started with Thonny MicroPython (Python) IDE:

If you want to program your ESP32 and ESP8266 with MicroPython firmware, it's very handy to use an IDE. you'll have your first LED blinking using MicroPython and Thonny IDE.



What is MicroPython?

MicroPython is a Python 3 programming language re-implementation targeted for microcontrollers and embedded systems. MicroPython is very similar to regular Python. Apart from a few exceptions, the language features of Python are also available in MicroPython. The most significant difference between Python and MicroPython is that MicroPython was designed to work under constrained Wi-Fi, and other communication protocol

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board
GP16	**LED**

PROGRAM

LED:

```
from machine import Pin
import time
LED = Pin(16, Pin.OUT)
while True:
    LED.value(1)
    time.sleep(1)
    LED.value(0)
    time.sleep(1)
```

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board (RGB)
GP16	**R**
GP17	**G**
GP18	**B**
GND	**COM**

RGB:

```
from machine import Pin
from time import sleep_ms,sleep
r=Pin(16,Pin.OUT)
y=Pin(17,Pin.OUT)
g=Pin(18,Pin.OUT)

while True:
    r.value(1)
    sleep_ms(1000)
    r.value(0)
    sleep_ms(1000)
    y.value(1)
    sleep(1)
    y.value(0)
    sleep(1)
    g.value(1)
    sleep(1)
    g.value(0)
    sleep(1)
```

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board
GP16	**LED**
GP15	**SW1**

SWITCH CONTROLLED LED:

```
from machine import Pin
from time import sleep
led=Pin(16,Pin.OUT)
sw=Pin(15,Pin.IN)
while True:
    bt=sw.value()
    if bt== True:
        print("LED ON")
        led.value(1)
        sleep(2)
        led.value (0)
        sleep(2)
        led.value (1)
        sleep(2)
        led.value(0)
        sleep(2)
    else:
        print("LED OFF")
    sleep(0.
```

EXP NO:9	INTERFACING SENSORS WITH RASPBERRY PI
DATE	

AIM:

To interface the IR sensor and Ultrasonic sensor with Raspberry Pico.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Thonny IDE	1
2	Raspberry Pi Pico Development Board	1
3	Jumper Wires	few
4	Micro USB Cable	1
5	IR Sensor	1
6	Ultrasonic sensor	1

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board	IR Sensor Module
GP16	BUZZER	-
GP15	-	OUT
-	5V	VCC
-	GND	GND

PROGRAM:

IR Sensor:

```
from machine import Pin
from time import sleep
buzzer=Pin(16,Pin.OUT)
ir=Pin(15,Pin.IN)
while True:
    ir_value=ir.value()
    if ir_value== True:
        print("Buzzer OFF")
        buzzer.value(0)
    else:
        print("Buzzer ON")
        buzzer.value (1)
    sleep(0.5)
```

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board	Ultrasonic Sensor Module
GP16	**BUZZER**	**-**
GP15	**-**	**ECHO**
GP14	**-**	**TRIG**
-	**5V**	**VCC**
-	**GND**	**GND**

ULTRASONIC SENSOR:

```
from machine import Pin, PWM
import utime
trigger = Pin(14, Pin.OUT)
echo = Pin(15, Pin.IN)
buzzer = Pin(16, Pin.OUT)
def measure_distance():
    trigger.low()
    utime.sleep_us(2)
    trigger.high()
    utime.sleep_us(5)
    trigger.low()
    while echo.value() == 0:
        signaloff = utime.ticks_us()
    while echo.value() == 1:
        signalon = utime.ticks_us()

    timepassed = signalon - signaloff
    distance = (timepassed * 0.0343) / 2
    return distance

 while True:
    dist = measure_distance()
    print(f"Distance : {dist} cm")
    if dist <= 10:
        buzzer.value(1)
        utime.sleep(0.01)
    else:
        buzzer.value(0)
        utime.sleep(0.01)
    utime.sleep(0.5)
```

EXP NO:9	COMMUNICATE BETWEEN ARDUINO AND RASPBERRY PI
DATE	

AIM:

To write and execute the program to Communicate between Arduino and Raspberry PI using any wireless medium (Bluetooth)

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Thonny IDE	1
2	Raspberry Pi Pico Development Board	1
3	Arduino Uno Development Board	1
4	Jumper Wires	few
5	Micro USB Cable	1
6	Bluetooth Module	2

PROGRAM:

MASTER

ARDUINO:

```
#include<SoftwareSerial.h>
SoftwareSerial mySerial(2,3); //rx,tx
void setup() {
        mySerial.begin(9600);
}

void loop() {
        mySerial.write('A');
        delay(1000);
        mySerial.write('B');
        delay(1000);
}
```

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board	Bluetooth Module
GP16	**LED**	**-**
VCC	**-**	**+5V**
GND	**-**	**GND**
GP1	**-**	**Tx**
GP0	**-**	**Rx**

SLAVE

RASPBERRY PI PICO

```
from machine import Pin, UART
uart = UART(0, 9600)
led = Pin(16, Pin.OUT)

while True:
  if uart.any() > 0:
    data = uart.read()
    print(data)
    if "A" in data:
      led.value(1)
      print('LED on \n')
      uart.write('LED on \n')
    elif "B" in data:
      led.value(0)
      print('LED off \n')
      uart.write('LED off \n')
```

EXP NO:11	CLOUD PLATFORM TO LOG THE DATA
DATE	

AIM:

To set up a cloud platform to log the data from IoT devices.

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No.	Software Requirements	Quantity
1	Blynk Platform	1

CLOUD PLATFORM-BLYNK:

Blynk is a smart platform that allows users to create their Internet of Things applications without the need for coding or electronics knowledge. It is based on the idea of physical programming & provides a platform to create and control devices where users can connect physical devices to the Internet and control them using a mobile app.

EXP NO:11	Log Data using Raspberry PI and upload it to the cloud platform
DATE	

AIM:

To write and execute the program Log Data using Raspberry PI and upload it to the cloud platform

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Thonny IDE	1
2	Raspberry Pi Pico Development Board	few
3	Jumper Wires	1
4	Micro USB Cable	1

CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board	LCD Module
-	5V	VCC
-	GND	GND
GP0	-	SDA
GP1	-	SCL

PROGRAM:

```
from machine import Pin, I2C, ADC
from utime import sleep_ms
from pico_i2c_lcd import I2cLcd
import time
import network
import BlynkLib

adc = machine.ADC(4)
i2c=I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
I2C_ADDR=i2c.scan()[0]
lcd=I2cLcd(i2c,I2C_ADDR,2,16)

wlan = network.WLAN()
wlan.active(True)
wlan.connect("Wifi_Username","Wifi_Password")

BLYNK_AUTH = 'Your_Token'

# connect the network
wait = 10
while wait > 0:
    if wlan.status() < 0 or wlan.status() >= 3:
        break
    wait -= 1
    print('waiting for connection...')
    time.sleep(1)

# Handle connection error
if wlan.status() != 3:
```

```
    raise RuntimeError('network connection failed')
else:
    print('connected')
    ip=wlan.ifconfig()[0]
    print('IP: ', ip)

"Connection to Blynk"
# Initialize Blynk
blynk = BlynkLib.Blynk(BLYNK_AUTH)

lcd.clear()
```

```
while True:
    ADC_voltage = adc.read_u16() * (3.3 / (65536))
    temperature_celcius = 27 - (ADC_voltage - 0.706)/0.001721
    temp_fahrenheit=32+(1.8*temperature_celcius)
    print("Temperature in C: {}".format(temperature_celcius))
    print("Temperature in F: {}".format(temp_fahrenheit))

    lcd.move_to(0,0)
    lcd.putstr("Temp:")
    lcd.putstr(str(round(temperature_celcius,2)))
    lcd.putstr("C ")
    lcd.move_to(0,1)
    lcd.putstr("Temp:")
    lcd.putstr(str(round(temp_fahrenheit,2)))
    lcd.putstr("F")
    time.sleep(5)

    blynk.virtual_write(3, temperature_celcius)
    blynk.virtual_write(4, temp_fahrenheit)
    blynk.log_event(temperature_celcius)

    blynk.run()

    time.sleep(5)
```

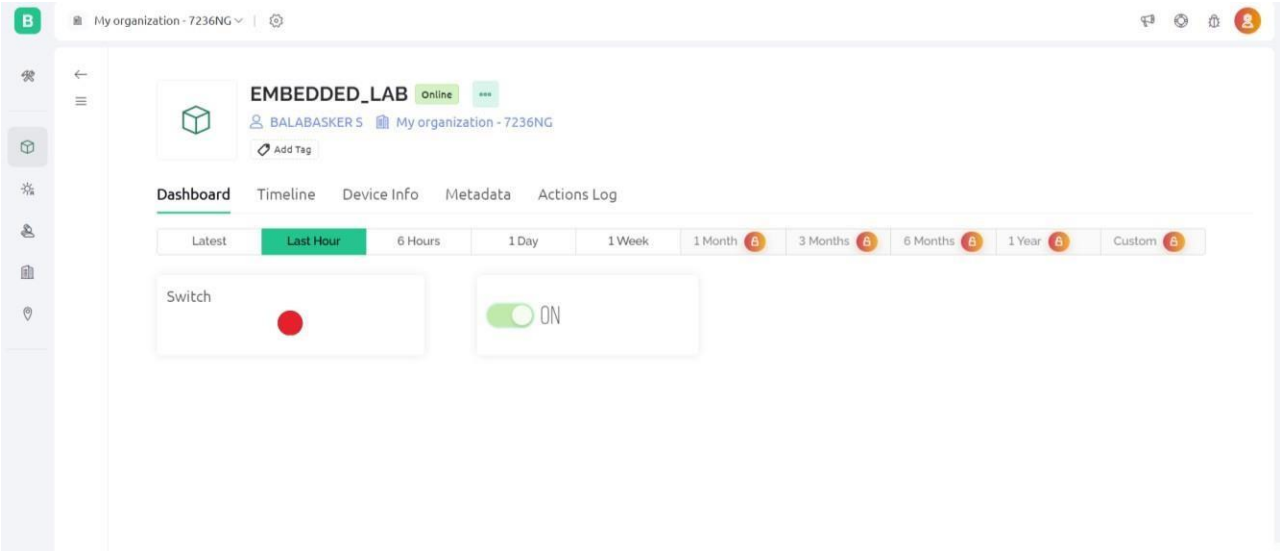
EXP NO:12	**Design an IOT-based system**
DATE	

AIM:

To design a Smart Home Automation IOT-based system

HARDWARE & SOFTWARE TOOLS REQUIRED:

S.No	Hardware & Software Requirements	Quantity
1	Thonny IDE	1
2	Raspberry Pi Pico Development Board	few
3	Jumper Wires	1
4	Micro USB Cable	1
5	LED or Relay	1



CONNECTIONS:

Raspberry Pi Pico Pin	Raspberry Pi Pico Development Board
GP16	LED 1

PROGRAM:

```
impo
rt
time
impo
rt
netw
ork
impo
rt
Blyn
kLib
from
machine
import Pin
led=Pin(16,
Pin.OUT)
```

```
wlan =
network.WL
AN()
wlan.active(T
rue)
wlan.connect("Wifi_Username","Wifi
_Password") BLYNK_AUTH =
'Your_Token'

# connect
the
network
wait = 10
while wait > 0:
    if wlan.status() < 0 or
        wlan.status() >= 3: break
    wait -= 1
    print('waiting for
    connection...')
    time.sleep(1)

# Handle
connection
error if
wlan.status()
!= 3:
    raise RuntimeError('network
connection failed') else:
    print('con
    nected')
    ip=wlan.i
    fconfig()
    [0]
    print('IP:
    ', ip)

"Connecti
on to
Blynk" #
Initialize
Blynk
blynk = BlynkLib.Blynk(BLYNK_AUTH)

# Register virtual pin
handler
```

```
@blynk.on("V0")
#virtual pin V0
def v0_write_handler(value):
    #read the value if int(value[0])
    == 1:
        led.value(1)
    #turn the led on
    else:
        led.value(0)
                                #tur
n the led off while
True:
    blynk.run()
```

BEYOND PROGRAMS

LCD DISPLAY:

```
from machine import Pin, I2C
from time import sleep
from pico_i2c_lcd import I2cLcd
i2c=I2C(0, sda=Pin(0), scl=Pin(1), freq=400000)
I2C_ADDR=i2c.scan()[0]
lcd=I2cLcd(i2c,I2C_ADDR,2,16)

while True:
    lcd.move_to(3,0)
    lcd.putstr("Ediylabs")
    sleep(5)
    lcd.clear()
```

Raspberry Pi Pico Development Board	16X2 LCD Display
5V	**VCC**
GND	**GND**
GP0	**SDA**
GP1	**SCL**

DHT 11 Sensor:

```
from machine import Pin,
I2C import utime as time
from dht import DHT11,
InvalidChecksum while True:
   time.sleep(1)
   pin = Pin(16, Pin.OUT,
   Pin.PULL_DOWN) sensor =
   DHT11(pin)
   t =
   (sensor.temperature) h
   = (sensor.humidity)
   print("Temperature:
   {}".format(sensor.temperature)) print("Humidity:
   {}".format(sensor.humidity)) time.sleep(2)
```

Connection

Raspberry Pi Pico Development Board	DHT11
5V	**VCC**
GND	**GND**
GP16	**DATA**

SERVO MOTOR:

```
from time import sleep
from machine import Pin, PWM

pwm = PWM(Pin(1))
pwm.freq(50)

while True:
    for position in
        range(1000,9000,50):
        pwm.duty_u16(position)
        sleep(0.01)
    for position in range(9000,1000,-
        50): pwm.duty_u16(position)
        sleep(0.01)
```

Connection

Raspberry Pi Pico Development Board	SERVOMOTOR
GND	BROWM
5V	RED
GP1	ORANGE

STEPPER MOTOR:

```
from machine import Pin
from time import sleep

IN1 =
Pin(12,Pin.OUT) IN2
= Pin(13,Pin.OUT)
IN3 =
Pin(14,Pin.OUT) IN4
= Pin(15,Pin.OUT)

pins = [IN1, IN2, IN3, IN4]

sequence = [[1,0,0,0],[0,1,0,0],[0,0,1,0],[0,0,0,1]]

while True:
    for step in sequence:
        for i in range(len(pins)):
            pins[i].value(step[i])
            sleep(0.001)
```

<u>Connection</u>

Raspberry Pi Pico Development Board	STEPPER MOTOR
5V	**+ (5V)**
GND	**- (GND)**
GP12	**IN1**
GP13	**IN2**
GP14	**IN3**
GP15	**IN4**